

PRISM

Implementation Specification

Line-level technical design for the full staged architecture (Stage 0 + Tier 1 + Tier 2), companion to the PRISM Project Proposal

iCog Labs — AGI Engineering Competition, Track A

Status: design specification, not yet run against a live PeTTa/PLN environment

1. Scope & Conventions

This document specifies every code change PRISM requires, at the file and function level. The Project Proposal answers what and why; this answers exactly how. Every MeTTa modification below references the real trueagi-io/PLN codebase as pasted and confirmed earlier in this project's development (tag v0.9-adjacent) — line numbers are given for orientation but should be re-checked against the exact commit in use before implementation, the same verification discipline applied throughout this project.

Notation: $STV(s,c)$ = Simple Truth Value (strength, confidence). $\$$ -prefix = MeTTa variable. Sentence = PLN's internal unit, (Sentence (<statement> (stv <s> <c>)) <evidence-stamp>).

2. Baseline: How PLN Works Today

2.1 The derivation loop — PLN.Derive

The verbatim current implementation, annotated by step:

```
(= (PLN.Derive $Tasks $Beliefs $steps $maxsteps $taskqueuesize $beliefqueuesize)
;; STEP 1: termination check
  (if (or (> $steps $maxsteps) (== $Tasks ()))
    ($Tasks $Beliefs)
    ;; STEP 2: select highest-priority task (BestCandidate + PriorityRank)
    (let (Sentence $x $Ev1) (BestCandidate PriorityRank () $Tasks)
      ;; STEP 3: derive $x against every belief (superpose $Beliefs)
      (let $derivations
        (collapse (superpose
          ((let* (((Sentence $y $Ev2) (superpose $Beliefs)))
            (if (StampDisjoint $Ev1 $Ev2)
              (let $stamp (msort (append $Ev1 $Ev2))
                (superpose ((case (|- $x $y) (...))
                  (case (|- $y $x) (...))))
              (empty))))
          (case (|- $x) (...))))
        ;; STEP 4: merge + prune to queue sizes (LimitSize + PriorityRankNeg)
        (progn (println! (SELECTED (Sentence $x $Ev1)))
          (PLN.Derive (LimitSize ...) $taskqueuesize)
            (LimitSize ...) $beliefqueuesize)
          (+ $steps 1) $maxsteps
          $taskqueuesize $beliefqueuesize))))))
```

2.2 Task selection and queue pruning

```
;; BestCandidate: generic linear-scan priority pick, takes the scoring
;; function as a first-class parameter – this is what makes the hook possible
(= (BestCandidate $evalFn $best $tuple)
  (if (== $tuple ()) $best
    (let* (($head (car-atom $tuple)) ($tail (cdr-atom $tuple)))
      (if (> ($evalFn $head) ($evalFn $best))
        (BestCandidate $evalFn $head $tail)
        (BestCandidate $evalFn $best $tail))))))

;; PriorityRank / PriorityRankNeg: PRISM's target – confidence only, today
(= (PriorityRank (Sentence ($x (stv $f $c)) $Ev1)) $c)
  (= (PriorityRank ()) -99999.0)
  (= (PriorityRankNeg (Sentence ($x (stv $f $c)) $Ev1)) (- 0.0 $c))
  (= (PriorityRankNeg ()) -99999.0))

;; LimitSize: trims a queue to size by repeatedly evicting the worst item
(= (LimitSize $L $size)
  (if (< (length $L) $size) $L
    (let $worst (BestCandidate PriorityRankNeg () $L)
      (LimitSize (exclude-item $worst $L) $size))))
```

Performance note, confirmed directly from this code: LimitSize calls BestCandidate once per evicted item. Trimming N items to K performs (N-K) linear scans, each re-evaluating the scoring function on every remaining item — $O((N-K) \times N)$ total calls. For N=100, K=10, that's roughly 8,100 scoring calls per single derivation step if uncached. This is the exact mechanism behind the FFI-caching risk in the proposal (§9) and is addressed directly in §6 below.

2.3 The goal-blindness bug in PLN.Query

```
(= (PLN.Query $Tasks $Beliefs $term $maxsteps $taskqueuesize $beliefqueuesize)
  (BestConfidenceCandidate
    (collapse (let ($TasksRet $BeliefsRet)
                  (PLN.Derive $Tasks $Beliefs $maxsteps $taskqueuesize $beliefqueuesize)
                  ;; ↑ $term is available here but never passed into PLN.Derive ↑
                  (case (superpose $BeliefsRet) (...)))))))
```

\$term (the query goal) is available in PLN.Query but is never forwarded into PLN.Derive — the derivation engine runs with zero knowledge of what's being sought, and all goal-filtering happens after the fact, by scanning the finished belief buffer. This is the exact bug the proposal's §6.1 goal-threading fix addresses.

3. Modification 1: Goal-Threading

The \$Goal parameter is added as an optional trailing argument via MeTTa's arity-based dispatch, per your direction: optional, with fallback, since some call sites won't have a goal available and a scorer call can itself fail.

3.1 Modified PLN.Query and PLN.Derive

```
;; PLN.Query now forwards $term as $Goal
(= (PLN.Query $Tasks $Beliefs $term $maxsteps $taskqueuesize $beliefqueuesize)
  (BestConfidenceCandidate
    (collapse (let ($TasksRet $BeliefsRet)
                  (PLN.Derive $Tasks $Beliefs $maxsteps $taskqueuesize $beliefqueuesize $term)
                  (case (superpose $BeliefsRet) (...)))))))

;; PLN.Derive: legacy 6-arg overload defaults $Goal to () — every existing
;; call site keeps working exactly as today, unchanged
(= (PLN.Derive $Tasks $Beliefs $steps $maxsteps $taskqueuesize $beliefqueuesize)
  (PLN.Derive $Tasks $Beliefs $steps $maxsteps $taskqueuesize $beliefqueuesize ()))

;; New 7-arg, goal-aware version
(= (PLN.Derive $Tasks $Beliefs $steps $maxsteps $taskqueuesize $beliefqueuesize $Goal)
  (if (or (> $steps $maxsteps) (== $Tasks ())) ($Tasks $Beliefs)
      (let (Sentence $x $Ev1) (BestCandidate (PriorityRankGoal $Goal) () $Tasks)
          (let $derivations (collapse (superpose (...))) ;; unchanged derivation logic
              (progn (println! (SELECTED (Sentence $x $Ev1)))
                     (PLN.Derive
                      (LimitSizeGoal (...) $taskqueuesize $Goal)
                      (LimitSizeGoal (...) $beliefqueuesize $Goal)
                      (+ $steps 1) $maxsteps $taskqueuesize $beliefqueuesize $Goal)))))))
```

3.2 The critical fix: PriorityRankGoal as a curried function

BestCandidate calls its scoring function with exactly one argument — (\$evalFn \$head). A two-argument replacement like (PriorityRank sentence goal) would never match this call and would silently break task selection entirely. The correct fix is partial application: (PriorityRankGoal \$Goal) evaluates to a function value that BestCandidate can then apply to one Sentence, exactly matching its existing calling convention:

```
;; (PriorityRankGoal $Goal) returns a function; BestCandidate calls it as
;; ($evalFn $head) = ((PriorityRankGoal $Goal) $head) — matches perfectly

(= ((PriorityRankGoal $Goal) ()) -99999.0)

(= ((PriorityRankGoal $Goal) (Sentence ($x (stv $f $c)) $Ev1))
```

```

(if (== $Goal ())
  $c ;; no goal supplied → undirected mode, raw confidence
  (let $score (py.call scorer score_candidate
    ((Sentence ($x (stv $f $c)) $Ev1) $Goal))
    (if (== $score Error)
      $c ;; scorer failure → graceful fallback, never halt derivation
      $score))))

(= ((PriorityRankNegGoal $Goal) ()) -99999.0)
(= ((PriorityRankNegGoal $Goal) (Sentence ($x (stv $f $c)) $Ev1))
  (- 0.0 ((PriorityRankGoal $Goal) (Sentence ($x (stv $f $c)) $Ev1))))

(= (LimitSizeGoal $L $size $Goal)
  (if (< (length $L) $size) $L
    (let $worst (BestCandidate (PriorityRankNegGoal $Goal) () $L)
      (LimitSizeGoal (exclude-item $worst $L) $size $Goal))))

```

3.3 What changes vs. what doesn't

Component	Change
PriorityRank / PriorityRankNeg	Replaced by curried PriorityRankGoal / PriorityRankNegGoal
LimitSize	Replaced by LimitSizeGoal (goal-aware, same eviction logic)
PLN.Derive	New 7-arg goal-aware version; legacy 6-arg overload preserved, unchanged behavior
PLN.Query	Forwards \$term as \$Goal; all convenience overloads unchanged
Inference rules (- clauses)	Untouched
Truth-value formulas (Truth_Deduction, etc.)	Untouched
BestCandidate	Untouched — already accepts any scoring function, which is what makes this fix possible
Existing examples and tests	Work unchanged via the legacy 6-arg overload

4. Stage 0: Indexed Premise Pre-Filter

Conditional, per proposal §6.2 — built only if the frontier-size measurement (proposal §7.3) shows it's actually needed. Unlike §3's fix, Stage 0 changes PLN.Derive's derivation body itself, not just the ranking functions outside it.

```

;; If built, replaces the unconditional (superpose $Beliefs) with:
(superpose
  (if (== $Goal ())
    $Beliefs ;; undirected
    (py.call stage0_index lookup_beliefs ($Goal $Beliefs)))) ;; indexed

```

Python-side index (structural key → candidate beliefs, backward from the goal):

```

class PremiseIndex:
    # Key: (link_type, concept) e.g. ("Inheritance", "Bird")
    # Value: list of Sentences whose statement matches that pattern
    def add(self, sentence):
        link_type, a, b = sentence.statement
        self._index[(link_type, a)].append(sentence)
        self._index[(link_type, b)].append(sentence)

    def lookup(self, goal):
        # For deduction (A→B, B→C ⊢ A→C): beliefs sharing the goal's
        # first arg, plus beliefs sharing its second arg

```

```
link_type, a, c = goal
return self._index[(link_type, a)] + self._index[(link_type, c)]
```

5. Tier 1: Fast Scorer

5.1 v1 — symbolic/structural heuristic (weeks 3–6, no training data required)

```
score_v1(candidate, goal) =  $\alpha$ ·overlap(S,G) +  $\beta$ ·expected_confidence +  $\delta$ · $\gamma$ ^depth

overlap(S, G) = |atoms(S)  $\cap$  atoms(G)| / |atoms(G)|           (Jaccard-style)
expected_confidence = c  (the candidate sentence's own truth-value confidence;
                        a conservative upper bound, since PLN's own formulas
                        – e.g. Truth_Deduction – never produce a conclusion
                        confidence exceeding the minimum premise confidence)
 $\gamma$ ^depth = geometric discount by derivation depth

Defaults:  $\alpha$ =0.5,  $\beta$ =0.3,  $\delta$ =0.2,  $\gamma$ =0.9 – tuned in week 7 against the
controlled synthetic domains (proposal §7.1)
```

5.2 v2 — trained embedding/GNN model (week 11)

Training data: (candidate, goal, outcome) triples logged during weeks 5–10, where outcome = 1 if the candidate was on the path that reached the goal, 0 if explored but discarded. Model: candidate/goal embeddings concatenated with STV features [strength, confidence, depth, queue-position], through a small MLP (128→64→1, ReLU, sigmoid output), trained with binary cross-entropy. Inference target: under 0.5ms per candidate on CPU. This is compared directly against v1 on held-out traces (proposal §8, week 11) — v2 replaces v1 only if it actually scores better, not by default.

5.3 Score caching (resolves the $O(N^2)$ FFI problem from §2.2)

```
class ScoreCache:
    # keyed on (sentence_hash, goal_hash); scores are immutable during
    # a single queue-sort since truth values don't change mid-sort
    def get_or_compute(self, sentence, goal, compute_fn):
        key = (hash(sentence), hash(goal))
        if key not in self._cache:
            self._cache[key] = compute_fn(sentence, goal)
        return self._cache[key]

# Main py.call entry point – called from MeTTa as:
# (py.call scorer score_candidate ($sentence $goal))
def score_candidate(sentence, goal):
    try:
        return _cache.get_or_compute(sentence, goal, _active_scorer) # v1 or v2
    except Exception as e:
        return "Error" # triggers the MeTTa-side fallback to raw confidence (§3.2)
```

6. Tier 2: Strategic LLM Reasoner

6.1 When it fires

Two precise conditions, not a judgment call: (1) the top Tier 1 score has stayed below τ_{stall} (default 0.2) for k_{stall} consecutive steps (default 5); or (2) the search's estimated remaining depth exceeds D_{max} (default 10).

6.2 Prompt structure

Goal, top-10 relevant known facts, and the last 5 derivation attempts are given to the LLM, which is asked to return a structured JSON response: a candidate subgoal (as a typed link expression), a suggested starting premise, and a brief rationale. The response is parsed and validated — it must type-check as a legal Sentence/goal atom — before it's trusted; a malformed or hallucinated subgoal is discarded, the same

anti-hallucination discipline the FORGE proposal applies to knowledge ingestion, applied here to subgoal generation instead.

6.3 Failure handling

An LLM call or JSON-parse failure returns None rather than raising — the stall counter is not reset, and the search continues with Tier 1 alone rather than blocking on a broken strategic layer.

7. The Search Algorithm

```
f(n) = g(n) + h(n)
g(n) =  $\sum_i \text{cost}(a_i)$ ,  $\text{cost}(a_i) = 1 - \text{confidence}(\text{conclusion of } a_i)$ 
      (high-confidence steps are cheap; low-confidence steps are expensive)
h(n) =  $1 - \text{score\_tier1}(\text{top\_candidate\_at\_n}, \text{Goal})$ 
      (high Tier 1 score  $\rightarrow$  low h(n)  $\rightarrow$  treated as close to the goal)
```

On the “A*” label, stated precisely: this is best-first search structured like A* ($f = g + h$), not admissible A* — $h(n)$ is a learned/heuristic estimate, not proven to never overestimate true distance to the goal, so A*'s classic optimality guarantee doesn't apply. This costs search efficiency in the worst case only, never correctness: PLN's own truth-value computation (§2) is completely unaffected by search order, so a suboptimal search order can at worst mean more steps to the same sound conclusion.

7.1 Search loop, condensed

```
def prism_search(tasks, beliefs, goal, config):
    pq = MinPriorityQueue(key=f)          # f(n) = g(n) + h(n)
    pq.push(initial_state(tasks, beliefs))
    while pq and steps < config.max_steps:
        state = pq.pop_min()
        if goal in state.beliefs: return state, trace          # done
        candidates = generate_candidates(state)                # unchanged PLN logic
        scored = [(cache.get_or_compute(c, goal, tier1.score), c) for c in candidates]
        if tier2.check_stall(max(s for s, _ in scored)):
            subgoal = tier2.decompose_goal(goal, state.beliefs, trace[-5:])
            if subgoal: backward_frontier.push(subgoal)          # §8
        for score, c in top_k(scored, config.top_k):
            new_state = apply(c, state)
            pq.push(new_state, f=new_state.g + (1 - score))
        if connects(state.beliefs, backward_frontier): return joined_trace # §8
    return None, trace    # goal not reached within budget
```

8. Bidirectional Search & Meet-in-the-Middle

The forward frontier is the standard Tier-1-guided PLN.Derive loop above. The backward frontier is seeded by Tier 2's subgoals on stall: for each subgoal, find which rules could produce it and which of their required premises are already known vs. still missing (the missing ones become further backward subgoals).

Termination:

```
meets_backward_frontier(forward_beliefs, backward_subgoals) =
     $\exists \text{ belief} \in \text{forward\_beliefs}, \text{subgoal} \in \text{backward\_subgoals} \text{ such that } \text{unify}(\text{belief}, \text{subgoal})$ 
```

When found, the full proof trace is reconstructed by chaining:

```
[initial premises]  $\rightarrow$  [forward derivations]  $\rightarrow$  [connection]  $\leftarrow$  [backward chain]  $\leftarrow$  [goal]
```

9. Testing & Validation

- `test_soundness.py` — runs the same task/belief set through unmodified PLN.Derive and the PRISM-guided version; asserts identical final conclusions and truth values, differing only in steps taken. Direct, automated version of proposal §7.3's soundness check — a release gate, not a one-time manual check.

- `test_fallback.py` — confirms `PriorityRankGoal`/`PriorityRankNegGoal` return raw confidence, unchanged, when `$Goal` is `()` or the scorer returns `Error`.
- `test_cache.py` — confirms cached and uncached scoring paths return identical results, and that the cache is cleared between derivation runs.
- `test_index_correctness.py` (if Stage 0 is built) — confirms the indexed lookup never excludes a belief that unindexed (superpose `$Beliefs`) would have included.
- Existing `ruletests/*.metta` — must all still pass unchanged via the legacy overloads, confirming zero regression to current PLN behavior.

10. File Manifest & Directory Layout

```

PLN/                                # trueagi-io/PLN (modified in-place)
├─ lib_pln.metta                    # Modified: §3 (goal-threading), §4 (Stage 0, if built)
├─ examples/                       # Unchanged – smoke tests
└─ ruletests/                      # Unchanged – regression tests, must still pass

prism/                             # New: PRISM implementation
├─ scorer.py                       # §5.3 – main py.call entry point
├─ cache.py                        # §5.3 – score memoization
├─ tier1_v1.py                     # §5.1 – symbolic/structural heuristic
├─ tier1_v2.py                     # §5.2 – trained model (week 11)
├─ tier2_llm.py                    # §6 – strategic LLM reasoner
├─ search.py                       # §7 – best-first search engine
├─ bidirectional.py                # §8 – backward chaining, meet-in-the-middle
├─ stage0_index.py                 # §4 – indexed pre-filter (conditional)
├─ benchmarks/                    # synthetic chain + KG multi-hop generators, metrics
├─ training/                      # week-11 trace collection + v2 training
└─ tests/                         # §9

```

11. Week-by-Week Build Order

Week	Deliverable	Pass/fail gate
1	py.call latency benchmark; <code>PriorityRankGoal</code> swap implemented	py.call round-trip < 1ms; all existing <code>ruletests/*.metta</code> still pass
2	Synthetic benchmark domains; unguided baseline numbers	Transitive chains $D \in [5, 20]$ run under unguided PLN; baseline waste ratios recorded
3–4	Tier 1 v1 symbolic heuristic; goal-threading live; Stage 0 if needed	<code>score_candidate</code> returns sensible scores; goal-aware PLN.Query matches baseline conclusions (soundness)
5–6	Best-first search loop; trace logging	Guided search on $D=5$ reaches goal in fewer steps than unguided; traces logged
7	Forward-only evaluation on controlled domains	Waste ratio reduced $\geq 20\%$ vs. baseline; not slower wall-clock than unguided
8	Tier 2 LLM integration; backward chaining; bidirectional termination	Tier 2 fires on stall; ≥ 1 bidirectional connection demonstrated
9	Scaling tests; cross-domain generalization	Sub-linear waste growth with <code>AtomSpace</code> size; cross-domain beats random baseline
10	Soundness verification; robustness/fallback testing	100% soundness across all benchmark runs; scorer failure degrades gracefully
11	Train Tier 1 v2; compare v1 vs. v2	v2 shortlist accuracy $\geq$ v1 on held-out traces, or v1 is kept

Week	Deliverable	Pass/fail gate
12	Final benchmark report; demo	All metrics documented; benchmark script reproducible end-to-end

12. Note on This Document's Provenance

This specification was developed jointly: the core architecture and math (§5–§7) trace back through this project's proposal revisions, and the code-level design (the curried `PriorityRankGoal` fix, the exact caching mechanism, the file manifest, and the week-by-week gates) was contributed by an independent technical review after the proposal's core claims had already been verified against the real PLN codebase. Both are reconciled here into one document. Nothing in this document has been run against a live PeTTa/PLN environment — that remains the next, separate step.